

爬虫（Python ver.）

前言

为什么选择python?

- 我个人有使用python写爬虫的经验
- python上手难度低，且目前对爬虫功能支持最全面

什么是爬虫

爬虫（Web Crawler），也称为网络爬虫、网页蜘蛛（Spider）或网络机器人（Bot），是一种自动化程序，用于系统地浏览和下载互联网上的网页内容。

基本概念

爬虫的核心思想是模拟人类浏览网页的行为，通过发送HTTP请求获取网页内容，然后解析和提取所需的信息。它能够：

- **自动访问网页**：无需人工干预，按照预设规则访问目标网站
- **提取有用信息**：从HTML、JSON、XML等格式中解析出需要的数据
- **批量处理**：一次性处理大量网页，提高数据收集效率
- **数据存储**：将提取的信息保存到数据库、文件等存储介质中

应用场景

爬虫在现代互联网中有着广泛的应用：

- **搜索引擎**：Google、百度等搜索引擎使用爬虫索引网页内容
- **数据分析**：收集电商价格、社交媒体数据、新闻资讯等
- **市场调研**：监控竞争对手信息、行业趋势分析
- **内容聚合**：新闻聚合网站、比价网站等
- **学术研究**：收集研究数据、文献信息等

关于robots.txt

目前大部分网站都带有robots.txt文件（具体查看方式为在网页链接后加上“/robots.txt”，如“www.bilibili.com/robots.txt”）。该文件约定了爬虫不能访问哪些路径、哪些爬虫能访问网页的哪些路径、哪些爬虫不能访问网页的哪些路径等信息。以b站的robots.txt为例：

```
User-agent: *
Disallow: /medialist/detail/
Disallow: /index.html

User-agent: Yisouspider
Allow: /
```

```
User-agent: Applebot
Allow: /

User-agent: bingbot
Allow: /

User-agent: Sogou inst spider
Allow: /

User-agent: Sogou web spider
Allow: /

User-agent: 360Spider
Allow: /

User-agent: Googlebot
Allow: /

User-agent: Baiduspider
Allow: /

User-agent: Bytespider
Allow: /

User-agent: PetalBot
Allow: /

User-agent: facebookexternalhit
Allow: /tbhx/hero

User-agent: Facebot
Allow: /tbhx/hero

User-agent: Twitterbot
Allow: /tbhx/hero

User-agent: *
Disallow: /
```

第一部分

```
User-agent: *
Disallow: /medialist/detail/
Disallow: /index.html
```

约定所有爬虫都不能访问 www.bilibili.com/medialist/detail/ 和 www.bilibili.com/index.html

第二部分，从 `User-agent: Yisouspider` 到 `User-agent: *` 之前，约定了允许访问网页的来自搜索引擎（或社交媒体等）的爬虫（这些爬虫需要遵循上一部分的约定）

第三部分

```
User-agent: *  
Disallow: /
```

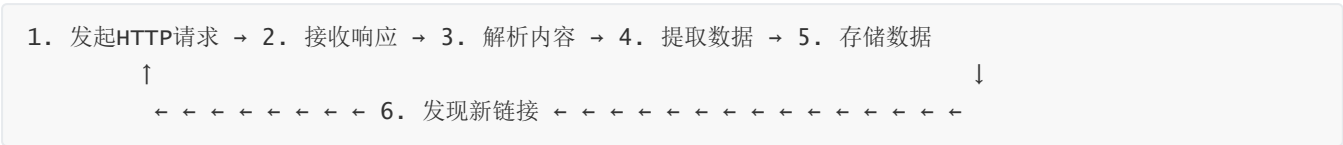
约定除了第二部分提到的爬虫外，其他所有爬虫都不被允许访问网页

当然，从技术上讲，尽管存在诸多反爬机制，网页其实无法彻底阻止爬虫的访问，因此robots.txt其实更像是一个“君子协定”，轻微的越界可能并不会带来什么后果，但如果做得太过分（比如抓取视频用于商业用途），轻则封禁IP，重则相关公司会通过法律手段维权。

爬虫原理

工作流程

爬虫的基本工作流程可以分为以下几个步骤：



详细步骤说明

1. 发起HTTP请求

- 构造请求头（User-Agent、Cookie等）
- 选择合适的HTTP方法（GET、POST等）
- 处理请求参数和表单数据

2. 接收响应

- 获取响应状态码（200、404、500等）
- 读取响应头信息
- 接收响应体内容

3. 解析内容

- HTML解析（使用DOM解析器）
- JSON/XML数据解析
- 文本内容提取

4. 提取数据

- 使用CSS选择器或XPath定位元素
- 正则表达式匹配特定模式
- 数据清洗和格式化

5. 存储数据

- 保存到文件（CSV、JSON、TXT等）
- 存储到数据库（MySQL、MongoDB等）

6. 发现新链接

- 从当前页面提取链接

- 添加到待爬取队列
- 去重处理

核心技术组件

1. HTTP客户端

负责与目标服务器进行通信：

- **请求库**：如Python的requests、urllib
- **会话管理**：维持登录状态、处理Cookie
- **代理支持**：通过代理服务器发送请求
- **重试机制**：处理网络异常和服务器错误

2. HTML解析器

用于解析和处理网页内容：

- **DOM解析器**：如BeautifulSoup、lxml
- **选择器引擎**：CSS选择器、XPath表达式
- **文本提取**：去除HTML标签，获取纯文本内容

3. 数据提取器

从解析后的内容中提取有用信息：

- **结构化数据**：表格、列表等规整数据
- **非结构化数据**：文章内容、评论等
- **元数据**：时间戳、作者信息等

4. 存储引擎

将提取的数据持久化保存：

- **文件存储**：适合小规模数据
- **数据库存储**：适合大规模结构化数据
- **缓存机制**：提高数据访问速度

爬虫怎么写（以网易云音乐网站为例）

基础版

[见base.py和base_cookie.py]

进阶版

[见pro_music.py与pro_comment.py]

高级版

本人尝试了多种方法试图写一份python代码用于演示，但是无奈能力有限，且随着时代发展，网页的反爬机制十分强大——网易云音乐在验证登录状态时非常严格（验证码+短信验证），故最终以失败告终。不过依旧欢迎各位如有兴趣去尝试一下，相信在尝试的过程中也会有所收获。下面列举了目前高级爬虫的若干核心技术：

1. 浏览器自动化技术

Selenium与WebDriver

- 通过控制真实浏览器（Chrome、Firefox等）来模拟人类操作
- 能够执行JavaScript，处理动态加载的内容
- 支持页面交互：点击、滚动、输入文本等
- 适用于SPA（单页应用）和AJAX重度依赖的网站

Headless浏览器

- 使用Headless Chrome或Playwright提升性能
- 在没有图形界面的服务器环境中运行
- 相比传统HTTP请求，资源消耗更大但兼容性更好

2. 反反爬策略

User-Agent轮换

- 维护常见浏览器的User-Agent池
- 随机或按规律切换，模拟不同用户设备
- 避免被识别为机器人程序

请求头伪造

- 模拟真实浏览器的完整请求头
- 包括Accept、Accept-Language、Accept-Encoding等
- 保持请求头的一致性和合理性

Cookie与Session管理

- 维持登录状态，模拟用户会话
- 处理CSRF Token等安全机制
- 合理管理Cookie的生命周期

3. 验证码处理方案

图像识别技术

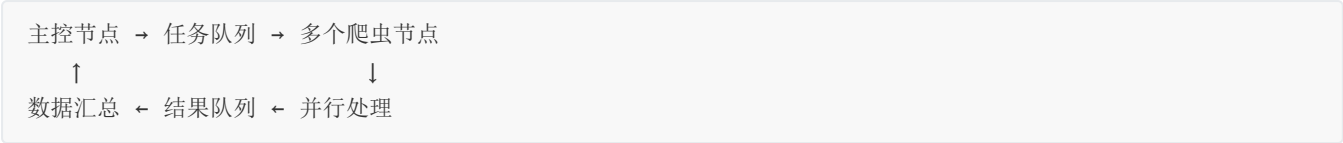
- OCR（光学字符识别）处理文字验证码
- 使用深度学习模型识别复杂图形验证码
- 集成第三方验证码识别服务

行为验证应对

- 滑动验证码：模拟鼠标轨迹和速度
- 点击验证：分析图像特征，精确定位目标
- 拼图验证：图像处理算法计算缺口位置

4. 分布式爬虫架构

任务分发机制



负载均衡

- 根据节点性能分配不同的任务量
- 动态调整爬取速度和并发数
- 故障转移和节点健康检查

5. 代理池技术

代理获取与验证

- 从免费/付费代理商获取IP
- 定期检测代理的可用性和速度
- 按地理位置、匿名程度分类管理

智能代理轮换

- 根据目标网站的封禁策略调整切换频率
- 监控代理IP的健康状态
- 实现故障自动切换和黑名单管理

6. 数据管道优化

增量更新策略

- 识别已爬取的内容，避免重复抓取
- 监控数据变化，仅更新有变动的部分
- 使用哈希值或时间戳进行去重

数据质量控制

- 实时验证数据的完整性和准确性
- 异常数据的自动识别和处理
- 数据格式标准化和清洗流程

7. 风险控制

频率控制算法

- 指数退避：遇到限制时逐渐降低请求频率
- 基于响应时间的自适应调整

监控与告警

- 实时监控爬虫状态和成功率
- 异常情况自动告警（IP被封、验证码增多等）
- 日志分析和性能指标统计

关于使用爬虫的几个建议

1. 用爬虫前先查看网页的robots.txt
2. 合理设置爬取频率，避免IP被封
3. 善用AI辅助

总结

1. 遵循robots.txt与相关法律法规
2. 遇到问题多问、多查
3. 多看、多写代码（包括用AI写）